

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
VIỆN ĐÀO TẠO QUỐC TẾ NIIE

MÔN KIẾN TRÚC PHẦN MỀM

BÁO CÁO KIẾN TRÚC PHẦN MỀM

Modular Monolith phân lớp, realtime và triển khai container

Thông tin	Nội dung
Giảng viên hướng dẫn	ThS. Trần Thanh Nhã
Đề tài	Nền tảng quản lý công việc và nhân sự doanh nghiệp TaskSyncEnterprise
Nhóm thực hiện	Scrum Team TaskSyncEnterprise - 5 thành viên
Phiên bản tài liệu	1.0 - Chuẩn hóa 2 Sprint

TP. Hồ Chí Minh, tháng 7 năm 2026

Tóm tắt điều hành

TaskSyncEnterprise là nền tảng web quản lý công việc, dự án và nhân sự doanh nghiệp. Giải pháp hiện tại lựa chọn Modular Monolith để giữ chi phí triển khai thấp, nhưng tổ chức mã nguồn theo ranh giới miền rõ ràng để có thể tách dịch vụ khi quy mô tăng.

Kết luận kiến trúc: SQL Server là nguồn dữ liệu chuẩn; Redis chỉ là cache-aside. RBAC được thực thi tại Backend. Domain event chỉ phát sau khi transaction commit thành công.

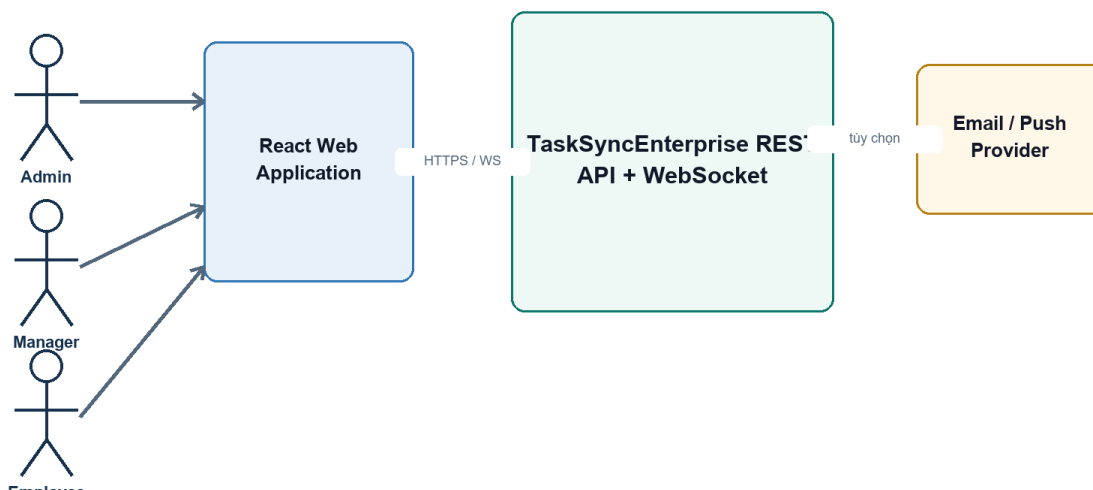
1. Bối cảnh và phạm vi

Hệ thống phục vụ ba nhóm người dùng chính: Administrator, Manager/Team Leader và Employee. Các miền nghiệp vụ gồm Organization, Project, Epic/Topic, Product Backlog, Sprint, Task, Collaboration, File, Vacation, Overtime, Notification và Audit.

- Frontend: React 19, Vite, React Router, TanStack Query và lớp dịch vụ Axios.
- Backend: Python/FastAPI, Pydantic, SQLAlchemy và Alembic.
- Dữ liệu: Microsoft SQL Server 2022; Redis 7 hỗ trợ cache và trạng thái vận hành.
- Realtime: WebSocket cập nhật các thay đổi nghiệp vụ mà không cần tải lại trang.
- Triển khai: Nginx, Docker Compose, persistent volume và overlay monitoring tùy chọn.

Sơ đồ ngữ cảnh hệ thống

Tác nhân và ranh giới của nền tảng TaskSyncEnterprise



Hình 1. Sơ đồ ngữ cảnh hệ thống TaskSyncEnterprise

2. Các động lực kiến trúc

Động lực	Yêu cầu kiến trúc	Cách đáp ứng
Dễ bảo trì	Ranh giới module rõ; phụ thuộc một chiều	Router → Service → CRUD → ORM
Bảo mật	Xác thực, RBAC và audit ở server	JWT/Session, policy backend, audit log
Realtime	Thay đổi hiển thị trên nhiều client	Domain event + WebSocket + query invalidation
Khả dụng	Cache không là điểm lỗi duy nhất	Cache-aside; SQL Server giữ dữ liệu chuẩn
Triển khai	Đơn giản cho nhóm 5 người	Modular Monolith trong Docker Compose
Mở rộng	Có thể tách tác vụ nặng	Ranh giới Notification, Reporting và AI

3. Phong cách kiến trúc

3.1. Modular Monolith

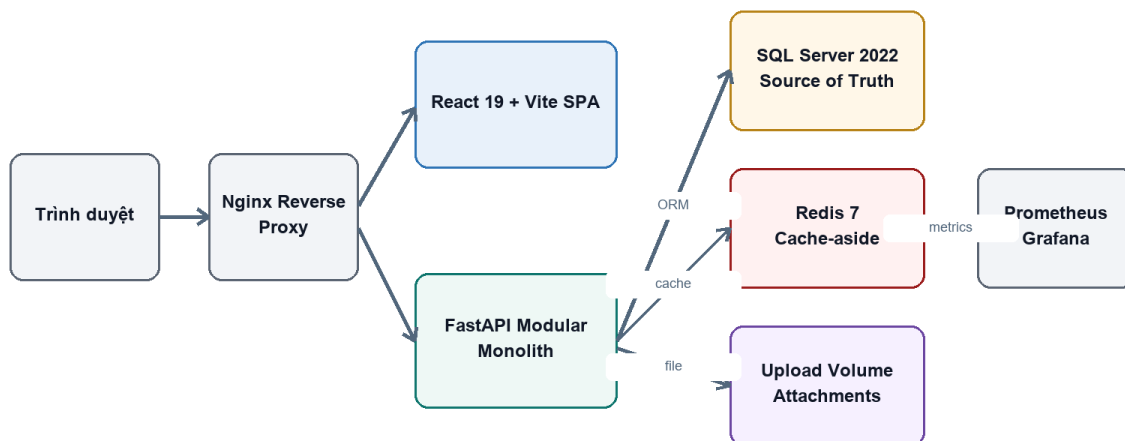
Kiến trúc được đóng gói như một ứng dụng Backend duy nhất nhưng chia module theo miền. Lựa chọn này phù hợp với quy mô nhóm, giảm độ phức tạp quan sát và đồng bộ dữ liệu so với microservices, đồng thời tránh gom toàn bộ logic vào một khối không cấu trúc.

3.2. Client - Server và kiến trúc phân lớp

React SPA giao tiếp REST API và WebSocket. Backend phân lớp để ràng buộc validation, authorization, business workflow và persistence. Frontend không phải là lớp bảo mật; việc ẩn nút chỉ cải thiện trải nghiệm người dùng.

Kiến trúc container

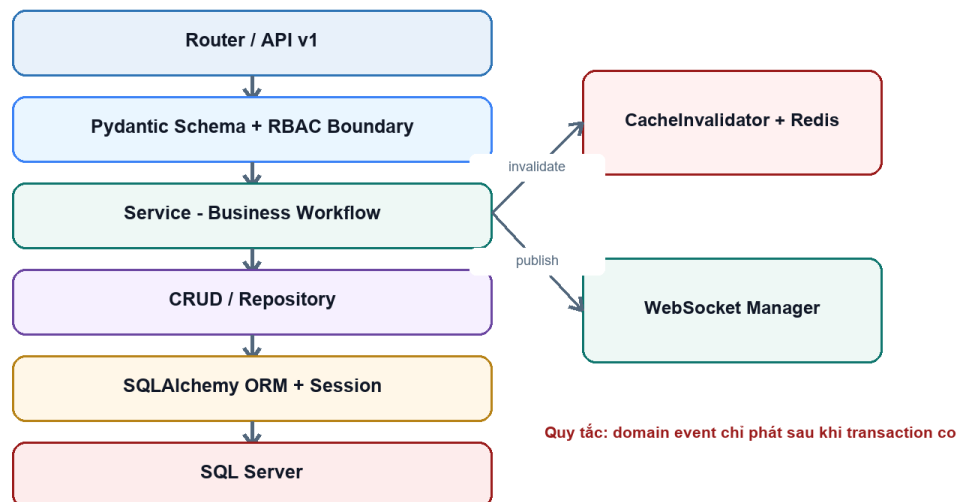
Các container triển khai và phụ thuộc vận hành



Hình 2. Kiến trúc container

Kiến trúc phân lớp Backend

Router → Service → CRUD → ORM → Database



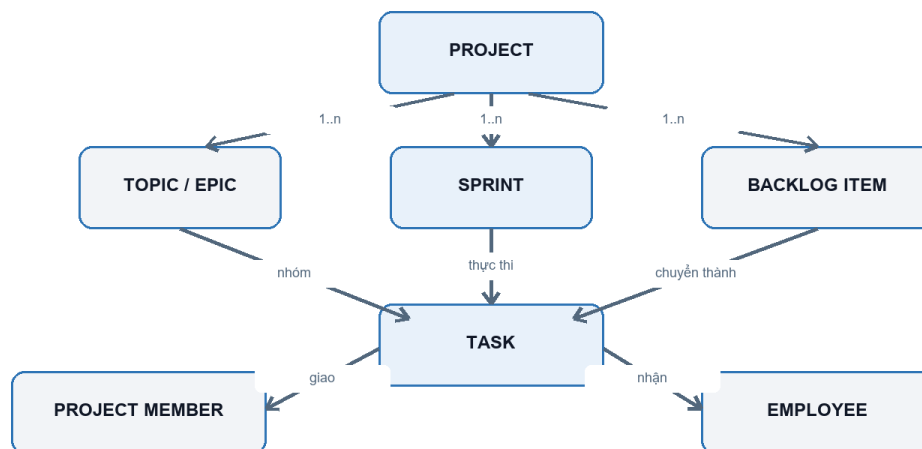
Hình 3. Kiến trúc phân lớp Backend

4. Mô hình miền và dữ liệu

Project là phạm vi liên kết Epic, Backlog Item, Sprint và Task. Backlog Item biểu diễn User Story có giá trị nghiệp vụ, ưu tiên và Story Point; khi được lựa chọn vào Sprint, item được phân rã thành Task thực thi.

Mô hình miền Project - Sprint - Task

Quan hệ cốt lõi giữa phạm vi dự án, backlog và người thực hiện



Hình 4. Mô hình miền Project - Sprint - Task

Invariant	Ý nghĩa
Đồng nhất Project	Task, Sprint, Backlog và Epic phải thuộc cùng một Project.
Quyền truy cập	Người được giao Task phải là thành viên có quyền truy cập Project.
Sprint đóng	Không thêm công việc vào Sprint đã hoàn tất nếu chưa reopen.
Capacity	Tổng Story Point cam kết không vượt capacity đã thống nhất.
Audit	Soft delete và trạng thái nghiệp vụ phải giữ được dấu vết thay đổi.

5. Luồng ghi dữ liệu và realtime

1. Client gửi POST/PATCH/DELETE tới API.
2. Backend kiểm tra schema, xác thực và quyền trước khi chạy workflow.
3. Service gọi CRUD/ORM trong transaction; SQL Server commit dữ liệu.
4. Sau commit, hệ thống phát domain event như task.changed hoặc vacation.changed.
5. Client khác nhận sự kiện, invalidate query key và refetch dữ liệu mới.

Nguyên tắc nhất quán: Không phát event trước commit. Nếu Redis ngừng hoạt động, nghiệp vụ chính vẫn đọc/ghi từ SQL Server; cache phải được xem là tối ưu hóa, không phải nguồn dữ liệu.

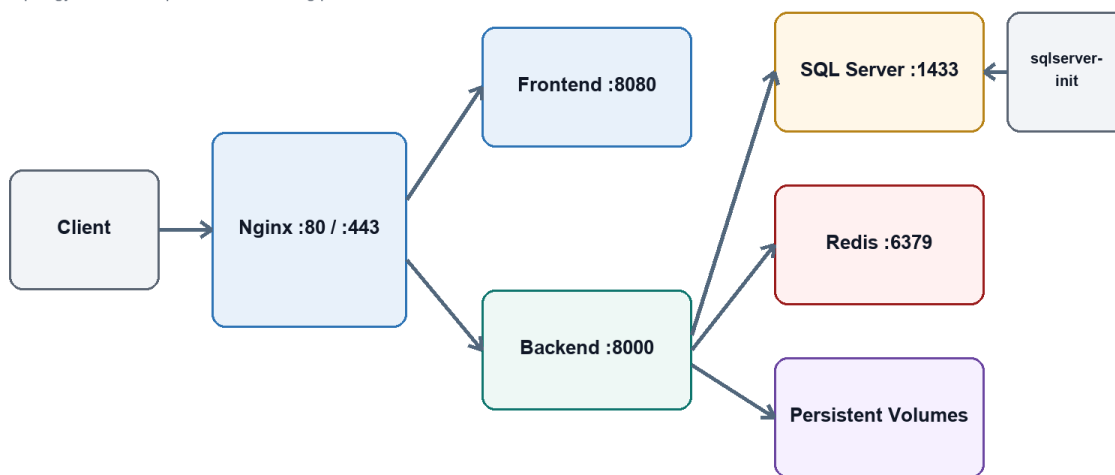
6. Bảo mật và kiểm soát truy cập

Lớp kiểm soát	Biện pháp
Transport	HTTPS tại Nginx; WebSocket qua kết nối được xác thực.
Identity	Login, access token/session, refresh và blacklist khi cần.
Authorization	RBAC tại Router/Service; kiểm tra phạm vi Project/Department.
Input	Pydantic validation, kiểm tra file, giới hạn kích thước và loại dữ liệu.
Persistence	Transaction, unique/FK constraints, migration Alembic.
Audit	Structured logging và audit trail cho thay đổi nhạy cảm.

7. Triển khai và vận hành

Sơ đồ triển khai

Topology Docker Compose cho môi trường production



Hình 5. Sơ đồ triển khai Docker Compose

Ở production, Nginx là điểm vào duy nhất. Frontend và Backend chạy trong container riêng; SQL Server và Redis có volume. Container sqlserver-init bảo đảm database tồn tại, trong khi Alembic chịu trách nhiệm cập nhật schema.

Trạng thái kiểm tra ngày 29/07/2026: 331 kiểm thử Backend và 22 kiểm thử Frontend được tài liệu audit ghi nhận đạt. Tuy nhiên Redis local đang DOWN, readiness trả 503, Docker Desktop engine chưa chạy và còn dependency advisory. Vì vậy tài liệu này không kết luận production-ready trước khi hoàn thành các điều kiện khắc phục.

8. Rủi ro, trade-off và hướng mở rộng

Rủi ro / trade-off	Xử lý đề xuất
Monolith tăng coupling khi module lớn	Giữ interface nội bộ; cấm truy cập chéo model tùy tiện.
Mất event giữa commit và publish	Áp dụng transactional outbox khi cần bảo đảm delivery.
Upload volume khó scale nhiều replica	Chuyển sang object storage khi triển khai phân tán.
Báo cáo/AI làm chậm API	Tách worker hoặc service bất đồng bộ.
Residual dependency risk	Theo dõi advisory và thay dependency khi có bản vá khả dụng.

9. Kết luận

Kiến trúc Modular Monolith đáp ứng tốt phạm vi đồ án và năng lực nhóm 5 người. Giá trị quan trọng nhất là ranh giới module, kiểm soát quyền tại Backend, SQL Server làm nguồn dữ liệu chuẩn và luồng realtime sau commit. Các hướng tách service chỉ nên thực hiện khi có số liệu tải và yêu cầu vận hành đủ rõ.

Tài liệu nguồn và đối chiếu

- docs/architecture/SYSTEM_ARCHITECTURE.md
- docs/architecture/DOMAIN_MODEL.md
- docs/reports/CONSOLIDATED_AUDIT.md
- Report/Tham khảo/Cac_bang_table.docx
- Report/Tham khảo/bao-cao-do-an-tasksynccenterprise (1).docx